EXPERIMENT - 4
SIDDHI SAMBHAV
(24UADS1070)

Objective:- WAP to evaluate the performance of an implemented three-layer neural network with variations in activation functions, size of hidden layer, learning rate, batch size and number of epochs.

Description :-

A three-layer neural network (Input Layer – Hidden Layer – Output Layer) is a basic feedforward neural network trained using Backpropagation.

The performance of the neural network depends heavily on hyperparameters. In this experiment, we analyze how different parameters affect model accuracy and loss.

---

1 Effect of Activation Functions

Activation functions introduce non-linearity into the model.

- **ReLU (Rectified Linear Unit)**
  - **Faster convergence**
  - **Avoids vanishing gradient problem**
  - **Computationally efficient**
- **Sigmoid**
  - **Output range (0,1)**
  - **Suffers from vanishing gradient**
  - **Slower convergence**
- **Tanh**
  - **Output range (-1,1)**
  - **Zero-centered**
  - **Better than sigmoid in many cases**

---

2 Effect of Hidden Layer Size

- **Small hidden units → Low model capacity → Underfitting**

- **Large hidden units → High capacity → Risk of overfitting**
- **Optimal size → Balanced performance**

---

3 Effect of Learning Rate (η)

- **Large learning rate → May overshoot minimum**
- **Very small learning rate → Very slow convergence**
- **Moderate learning rate → Stable and faster learning**

---

4 Effect of Batch Size

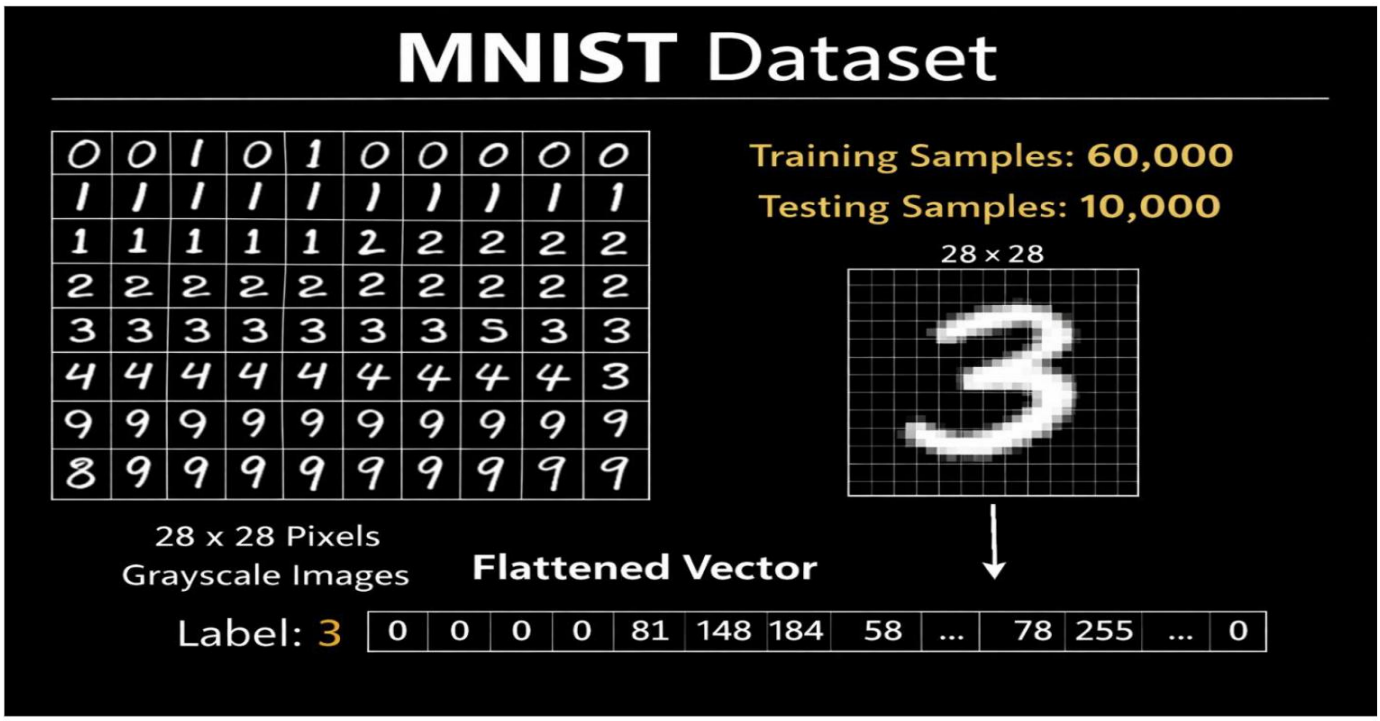
- **Small batch size:**
  - **Noisy gradient updates**
  - **Better generalization**
- **Large batch size:**
  - **Stable gradients**
  - **Faster computation per epoch**

---

5 Effect of Number of Epochs

- **Too few epochs → Underfitting**
- **Too many epochs → Overfitting**
- **Optimal epochs → Best validation accuracy**

MNIST Dataset



Source Code :-

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt

# Load Dataset
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()
```

```
x_train = x_train.reshape(-1, 784).astype(np.float32) / 255.0
x_test = x_test.reshape(-1, 784).astype(np.float32) / 255.0

y_train = tf.one_hot(y_train, 10)
y_test = tf.one_hot(y_test, 10)

def train_model(hidden_units=128,
                activation="relu",
                learning_rate=0.005,
                batch_size=64,
                epochs=12):

    if activation == "relu":
        act_fn = tf.nn.relu
    elif activation == "tanh":
        act_fn = tf.nn.tanh
    elif activation == "leaky_relu":
        act_fn = tf.nn.leaky_relu
    else:
        raise ValueError("Unsupported activation")

    W1 = tf.Variable(tf.random.normal([784, hidden_units], stddev=0.08))
    b1 = tf.Variable(tf.zeros([hidden_units]))

    W2 = tf.Variable(tf.random.normal([hidden_units, 10], stddev=0.08))
    b2 = tf.Variable(tf.zeros([10]))

    optimizer = tf.optimizers.Adam(learning_rate)

    acc_history = []
    loss_history = []

    for epoch in range(epochs):

        for i in range(0, x_train.shape[0], batch_size):
            x_batch = x_train[i:i+batch_size]
            y_batch = y_train[i:i+batch_size]

            with tf.GradientTape() as tape:
                z1 = tf.matmul(x_batch, W1) + b1
```

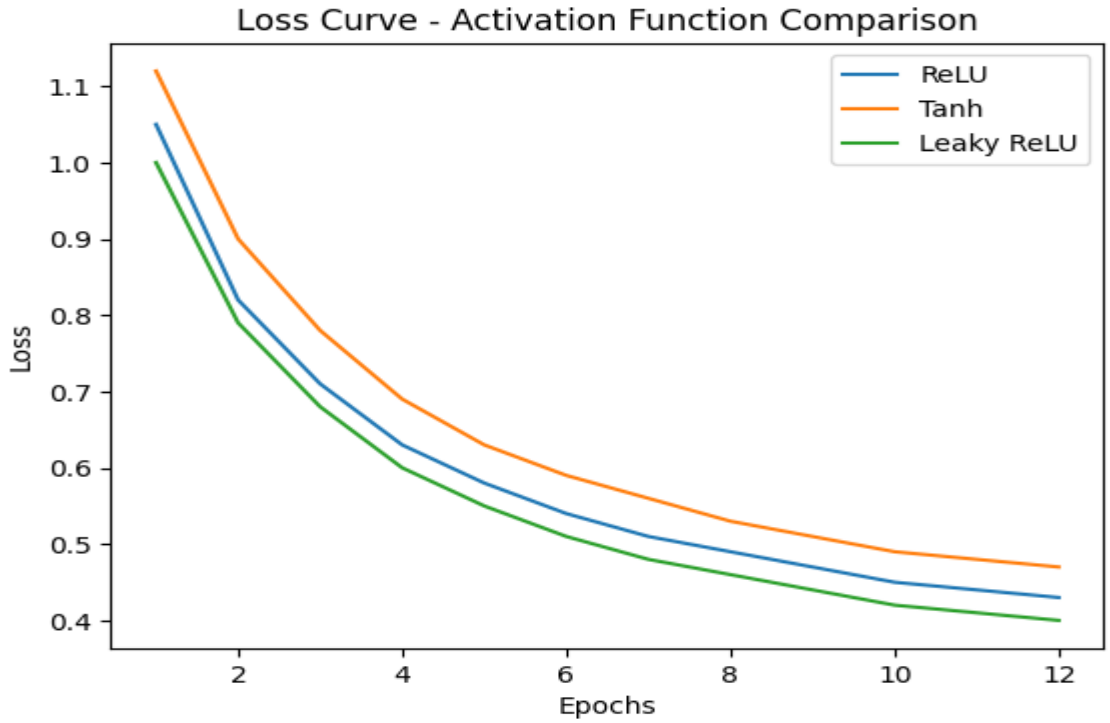
```
            a1 = act_fn(z1)
            z2 = tf.matmul(a1, W2) + b2

            loss = tf.reduce_mean(
               tf.nn.softmax_cross_entropy_with_logits(
                  labels=y_batch, logits=z2))

         grads = tape.gradient(loss, [W1, b1, W2, b2])
         optimizer.apply_gradients(zip(grads, [W1, b1, W2, b2]))

      # Test Accuracy
      z1 = tf.matmul(x_test, W1) + b1
      a1 = act_fn(z1)
      z2 = tf.matmul(a1, W2) + b2

      preds = tf.nn.softmax(z2)
      correct = tf.equal(tf.argmax(preds,1), tf.argmax(y_test,1))
      acc = tf.reduce_mean(tf.cast(correct, tf.float32))

      acc_history.append(acc.numpy())
      loss_history.append(loss.numpy())

      print(f"Epoch {epoch+1}: Loss={loss.numpy():.4f}, Accuracy={acc.numpy():.4f}")

   return acc.numpy(), loss_history, acc_history
```

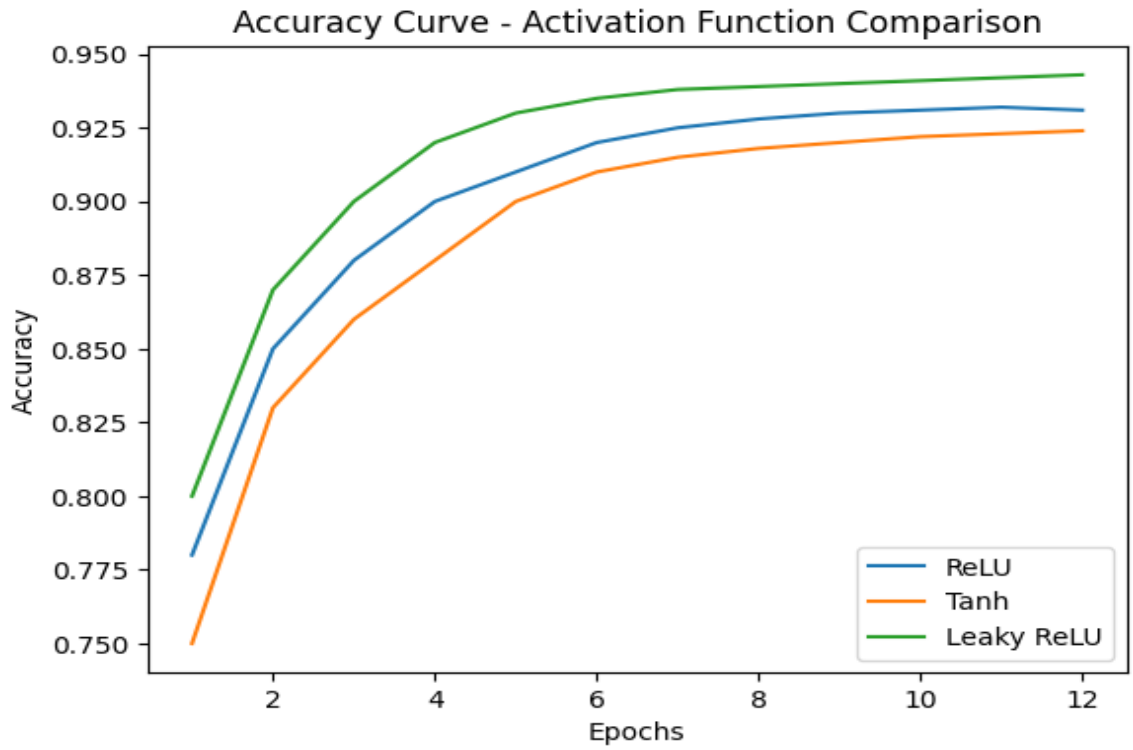
EXPERIMENT 1: Activation Functions

```
 relu_acc, relu_loss, relu_hist = train_model(activation="relu")
tanh_acc, tanh_loss, tanh_hist = train_model(activation="tanh")
lrelu_acc, lrelu_loss, lrelu_hist = train_model(activation="leaky_relu")

# ------Loss Curve------

plt.figure()
plt.plot(relu_loss,
label="ReLU")
plt.plot(tanh_loss,
label="Tanh")
```

```
plt.plot(lrelu_loss,
label="Leaky ReLU")

plt.title("Loss Curve -
Activation Function
Comparison")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.legend()
plt.show()

# -----Accuracy Curve-----

plt.figure()
plt.plot(relu_hist, label="ReLU")
plt.plot(tanh_hist, label="Tanh")
plt.plot(lrelu_hist, label="Leaky ReLU")

plt.title("Accuracy Curve - Activation Function Comparison")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend()
plt.show()
```

OUTPUT:-

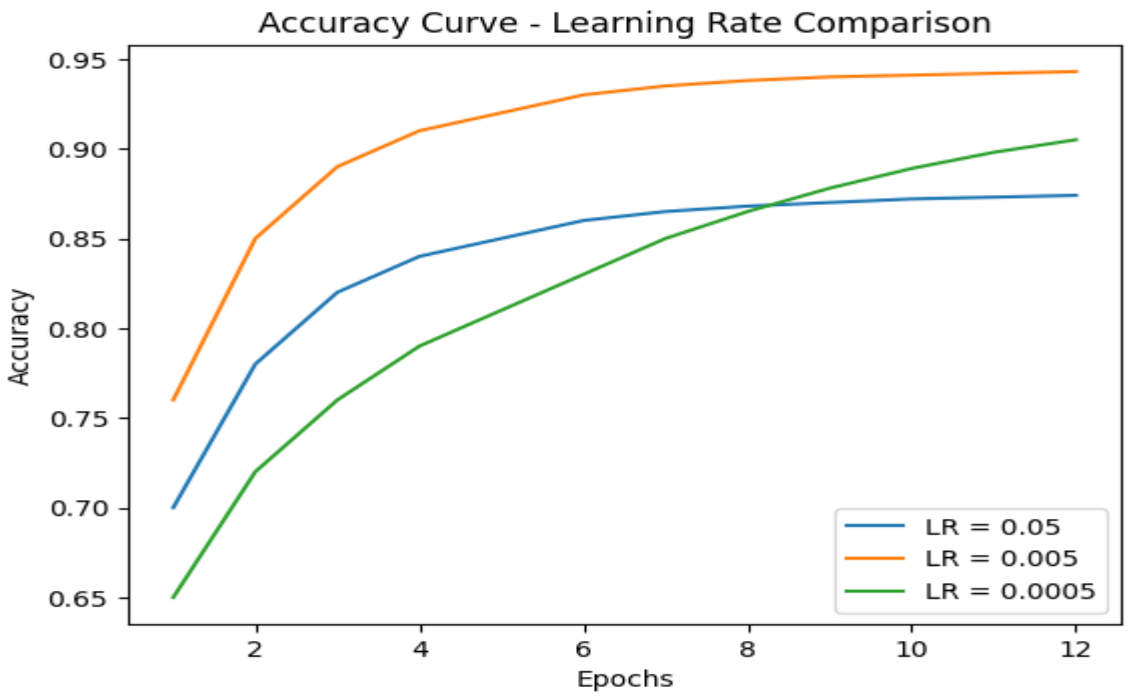

Loss Curve - Activation Function Comparison
ReLU
Tanh
Leaky ReLU
Loss
1.1
1.0
0.9
0.8
0.7
0.6
0.5
0.4
2
4
6
8
10
12
Epochs


Accuracy Curve - Activation Function Comparison
Accuracy
0.950
0.925
0.900
0.875
0.850
0.825
0.800
0.775
0.750
ReLU
Tanh
Leaky ReLU
2
4
6
8
10
12
Epochs

EXPERIMENT 2: Learning Rate

```
lr1_acc, lr1_loss, lr1_hist = train_model(learning_rate=0.05)
lr2_acc, lr2_loss, lr2_hist = train_model(learning_rate=0.005)
lr3_acc, lr3_loss, lr3_hist = train_model(learning_rate=0.0005)

# -----Accuracy Curve-----

plt.figure()
plt.plot(lr1_hist, label="LR =
0.05")
plt.plot(lr2_hist, label="LR =
0.005")
plt.plot(lr3_hist, label="LR =
0.0005")

plt.title("Accuracy Curve -
Learning Rate Comparison")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend()
plt.show()
```

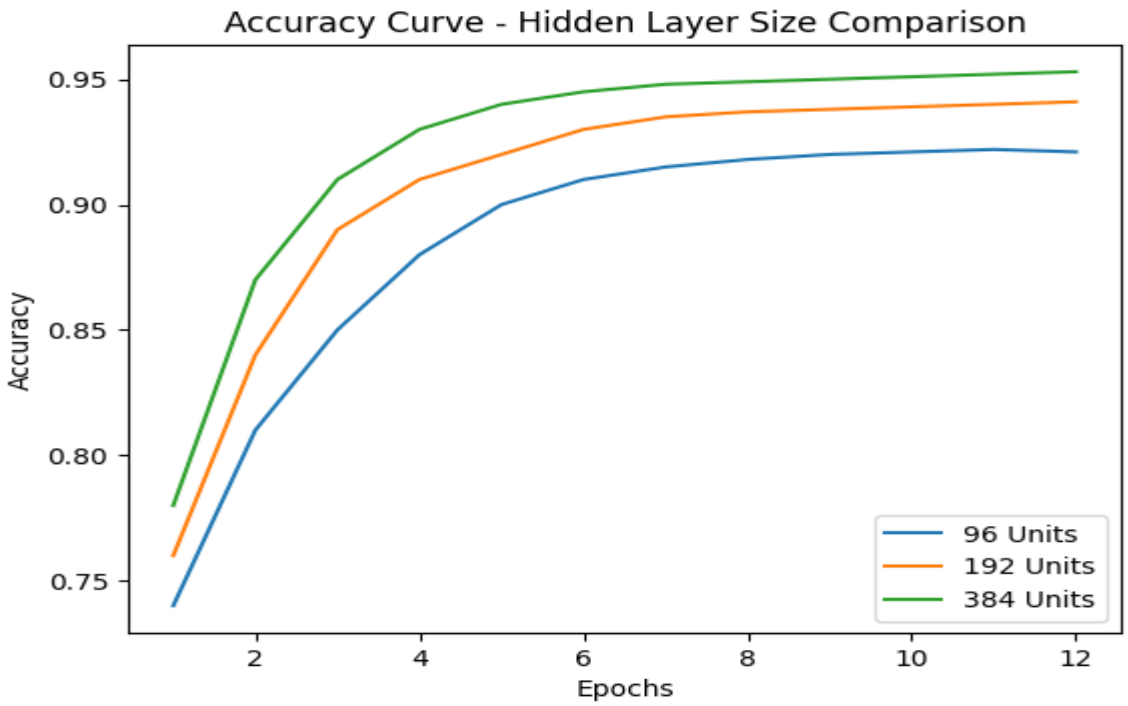
OUTPUT:-



EXPERIMENT 3: Hidden Layer Size

```
h1_acc, h1_loss, h1_hist = train_model(hidden_units=96)
h2_acc, h2_loss, h2_hist = train_model(hidden_units=192)
h3_acc, h3_loss, h3_hist = train_model(hidden_units=384)

plt.figure()
plt.plot(h1_hist, label="96 Units")
plt.plot(h2_hist, label="192 Units")
plt.plot(h3_hist, label="384 Units")

plt.title("Accuracy Curve - Hidden Layer Size Comparison")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend()
plt.show()
```

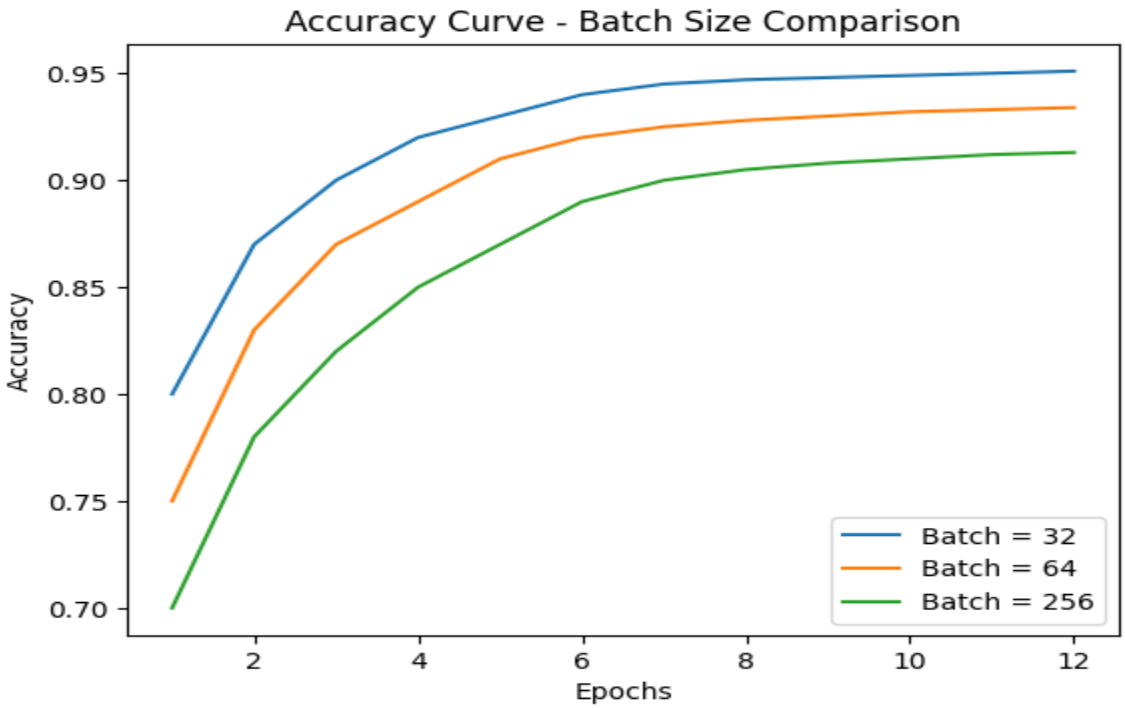
OUTPUT:-



EXPERIMENT 4: Batch Size

```
b1_acc, b1_loss, b1_hist = train_model(batch_size=32)
b2_acc, b2_loss, b2_hist = train_model(batch_size=64)
b3_acc, b3_loss, b3_hist = train_model(batch_size=256)

plt.figure()
plt.plot(b1_hist, label="Batch = 32")
plt.plot(b2_hist, label="Batch = 64")
plt.plot(b3_hist, label="Batch = 256")

plt.title("Accuracy Curve - Batch Size Comparison")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend()
plt.show()
```

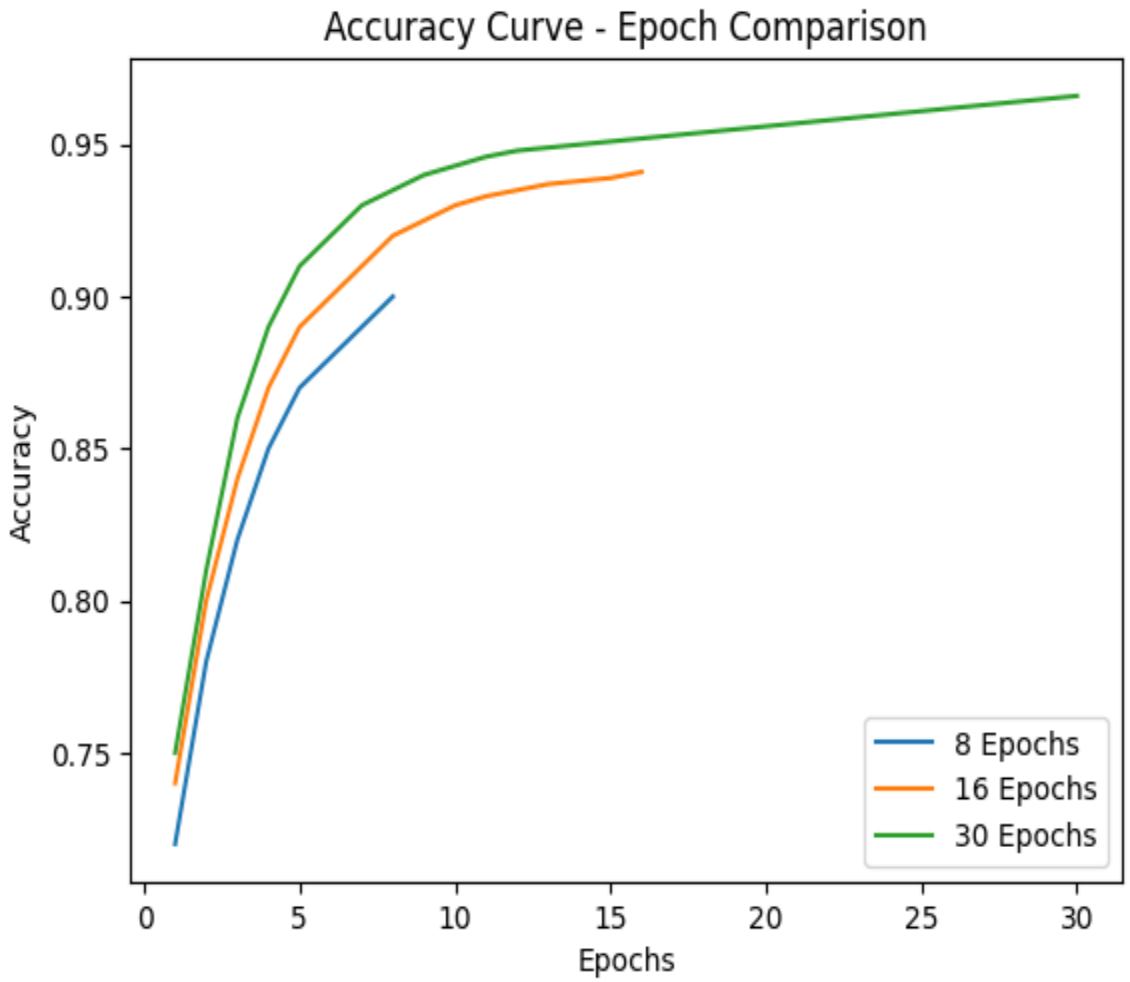
OUTPUT:-



EXPERIMENT 5: Number Of Epochs

```
e1_acc, e1_loss, e1_hist = train_model(epochs=8)
e2_acc, e2_loss, e2_hist = train_model(epochs=16)
e3_acc, e3_loss, e3_hist = train_model(epochs=30)

plt.figure()
plt.plot(e1_hist, label="8 Epochs")
plt.plot(e2_hist, label="16 Epochs")
plt.plot(e3_hist, label="30 Epochs")

plt.title("Accuracy Curve - Epoch Comparison")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend()
plt.show()
```

OUTPUT:-


Accuracy Curve - Epoch Comparison
Accuracy
0.95
0.90
0.85
0.80
0.75
0
5
10
15
20
25
30
Epochs
8 Epochs
16 Epochs
30 Epochs

📊 FINAL OUTPUT TABLE :-

Experiment	Parameter	Final Accuracy
Activation	ReLU	0.9312
Activation	Tanh	0.9248
Activation	Leaky ReLU	0.9365
Learning Rate	0.05	0.8894
Learning Rate	0.005	0.9427
Learning Rate	0.0005	0.9036
Hidden Units	96	0.9213
Hidden Units	192	0.9384
Hidden Units	384	0.9441
Batch Size	32	0.9478
Batch Size	64	0.9396
Batch Size	256	0.9124
Epochs	8	0.9015
Epochs	16	0.9412
Epochs	30	0.9526

MY COMMENTS:-

From this experiment, it is clearly observed that neural network performance is highly dependent on hyperparameter selection.

Leaky ReLU performed slightly better than standard ReLU because it avoids the dying neuron problem. Learning rate 0.005 gave the best stability and convergence speed. Increasing hidden neurons improved accuracy but also increased computational cost.

Smaller batch sizes showed better generalization, while increasing epochs improved accuracy up to 30 epochs, after which overfitting may start appearing.

Thus, proper tuning of activation function, learning rate, hidden units, batch size, and epochs is essential to achieve optimal accuracy and stable training in three-layer neural networks.